SE Experiment 4

Team:
Stavan Desai - 2024300042
Dev Gaglani - 2024300057

Batch: A3 / AC

Aim:

To design and represent the dynamic behavior of the **Inventory and Stock Management System (ISMS)** using UML **Sequence and Collaboration (Communication) Diagrams** for three important use cases. The diagrams are derived from the classes, relationships, and methods identified in the class diagram and the functional requirements specified in the SRS. The sequence diagrams show the chronological interaction among actors and system objects, while the collaboration diagrams emphasize the relationships and message flow among the participating objects. The diagrams are designed to remain consistent with the class diagram.

Theory:

Sequence Diagram

A Sequence Diagram is a UML behavioral diagram used to describe how actors and objects interact with one another to accomplish a particular use case. It represents the interaction in a time-ordered manner, with the sequence of messages shown from top to bottom.

The major elements of a sequence diagram are:

- Actor: Represents an external user or entity interacting with the system.
- Lifeline: Represents an object or participant involved in the interaction.
- Message: Represents a request, method call, or communication between participants.
- Activation: Represents the period during which an object is actively executing an operation.
- Return Message: Represents the response produced after execution of an operation.
- Guard Condition: Specifies a condition under which a particular interaction takes place.
- Combined Fragments: Represent conditional or repetitive behavior. Common fragments include:
    - `alt` – represents alternative flows such as success/failure or approve/reject.
    - `opt` – represents an optional flow.
    - `loop` – represents repeated execution of an interaction.

For the ISMS, sequence diagrams are useful because the SRS defines several stimulus-response sequences, such as logging a physical sale, processing a purchase order, authenticating a user, updating stock, and generating alerts. For example, the SRS states that when Sales Personnel log a physical sale, the system deducts the sold quantity from stock and generates an invoice.

A sequence diagram must correspond to the class diagram. Therefore, messages representing operations should map to methods belonging to the relevant classes. The ISMS class model contains entities such as `User`, `Product`, `StockRecord`, `PurchaseOrder`, `SalesOrder`, `Forecast`, and `Notification`, which form the basis for the interactions.

Collaboration / Communication Diagram

A Collaboration Diagram, also known as a Communication Diagram, represents the interaction between objects with greater emphasis on their structural relationships and message exchange rather than the exact vertical timeline.

It consists mainly of:

- Objects or participants involved in the use case.
- Links showing communication relationships between objects.
- Numbered messages showing the order in which operations are performed.
- Conditions or iterations associated with messages where required.

While a sequence diagram answers “when does each interaction occur?”, a collaboration diagram focuses more on “which objects communicate with each other and what messages do they exchange?”

Both diagrams describe the same dynamic behavior from different perspectives. In this assignment, the participating objects and operations are derived from the class diagram, while the scenarios and system behavior are based on the requirements defined in the SRS.

The ISMS is particularly suitable for these diagrams because its major functions involve interaction among users, products, stock records, purchase orders, sales orders, forecasting, and notifications. The SRS identifies seven major feature areas, including authentication, inventory and omnichannel synchronization, supplier management, purchase order management, sales management, predictive analytics, and multimodal notifications.

Role of Fragments

Fragments are important in sequence diagrams because real-world system behavior is rarely completely linear. The ISMS contains several conditional flows that can be represented using fragments.

For example:

- An order may be approved or rejected.
- A purchase order may be fully or partially received.
- A sale may be completed normally or followed by a return/cancellation.
- A login attempt may succeed or fail.
- A voice command may be accepted or may fall back to standard UI input when confidence is low.

The SRS explicitly defines such alternatives. For example, a purchase order can move from Pending Approval to Approved when accepted, or return to Draft when rejected. It can also move to Partially Received when only part of the shipment is received.

Problem Description:

The Inventory and Stock Management System (ISMS) is a unified web-based system designed to replace fragmented manual inventory processes such as spreadsheets, paper stock registers, and disconnected marketplace dashboards. It provides a common platform for Administrators, Managers, Warehouse Staff, and Sales Personnel to manage inventory, suppliers, purchase orders, and sales.

The system must support interactions across multiple business activities. These include secure authentication, stock management, purchase order processing, sales tracking, forecasting, and notifications. The SRS also specifies integration with marketplaces, speech-to-text/text-to-speech services, forecasting services, payment or invoicing gateways, and e-mail/SMS services.

Because these operations involve several users and objects interacting in a specific order, a static class diagram alone is not sufficient to describe system behavior. Sequence and collaboration diagrams are therefore required to model the dynamic interaction between system participants.

For this assignment, three representative use cases are considered:

1. Physical Sales Processing – Sales Personnel records a physical sale, the sales order is created, stock is updated, and an invoice is generated.
2. Purchase Order Management – A purchase order is created, approved or rejected, and subsequently received by Warehouse Staff.
3. Authentication – A user attempts to log in, with alternate behavior for successful and unsuccessful authentication.

These use cases cover important interactions among the classes and methods identified in the class diagram and are also directly supported by the functional requirements in the SRS. The sales requirements specify physical sales logging, automatic stock deduction, invoice generation, and return processing. The purchase-order requirements specify creation, approval, rejection, receipt confirmation, partial receipt, and overdue tracking. Authentication requirements specify login, password reset, role-based access, and failed-login handling.

Implementation:

The Sequence and Collaboration Diagrams for the Inventory and Stock Management System (ISMS) were implemented for three major use cases: Inventory Management, Manage Purchase Order, and Process Sales. The interactions were derived from the classes and methods identified in the class diagram and the corresponding functional requirements defined in the SRS. The diagrams therefore maintain consistency between the structural and behavioral

models of the system. The SRS identifies inventory management, purchase order management, and sales tracking as major functions of the ISMS.

4.1 Inventory Management

The Inventory Management use case represents the management of stock records and products within the system. Warehouse Staff can perform inventory operations through the standard interface or hands-free voice commands. The SRS specifies that the system supports creation, updating, deletion, and searching of inventory records, along with real-time synchronization of stock changes across connected sales channels.

The sequence diagram models the interaction between the user and the relevant inventory classes. Operations such as create(), update(), delete(), update_stock(), deduct_stock(), and realtime_sync() correspond to the methods represented in the class diagram. Conditional fragments are used to represent alternative flows, such as successful stock modification or a low-confidence voice command that requires another input method.

4.2 Manage Purchase Order

The Manage Purchase Order use case represents the complete handling of purchase orders, beginning with their creation and continuing through approval, rejection, receipt, and overdue tracking. According to the SRS, a Manager creates a purchase order by specifying the supplier, products, quantities, and expected delivery date. The order must subsequently be approved according to the configured approval limits before being transmitted to the supplier.

The sequence diagram uses the PurchaseOrder class and its corresponding methods such as create(), approve(), reject(), receive_confirm(), and flag_overdue(). Manager and Warehouse Staff participate in the interaction according to their defined responsibilities. An alt fragment is used to represent the alternative approval and rejection paths, while an opt fragment can represent optional receipt or overdue processing.

The diagram also reflects the purchase-order lifecycle defined in the SRS. An approved order may become Partially Received when the received quantity is less than the ordered quantity, or Received when all outstanding quantities are received.

4.3 Process Sales

The Process Sales use case represents the processing of physical and digital sales. The SRS specifies that Sales Personnel can record physical sales, while marketplace webhook events can be processed for digital orders. The system creates or updates the corresponding sales order, deducts stock upon order confirmation, and generates an invoice for completed sales. Returns or cancellations reverse the associated stock deduction.

The sequence diagram uses the SalesOrder and Product classes along with the Sales Personnel actor. Methods such as create_order(), process_webhook(), process_return(), and generate_invoice() are used according to the class diagram. The alt fragment represents the

different sales processing paths, such as physical and digital sales, while an opt fragment can represent an optional return or cancellation.

The implementation also considers the insufficient-stock condition specified in the SRS. When the requested quantity exceeds available stock, the order line is rejected and the responsible Manager is notified.

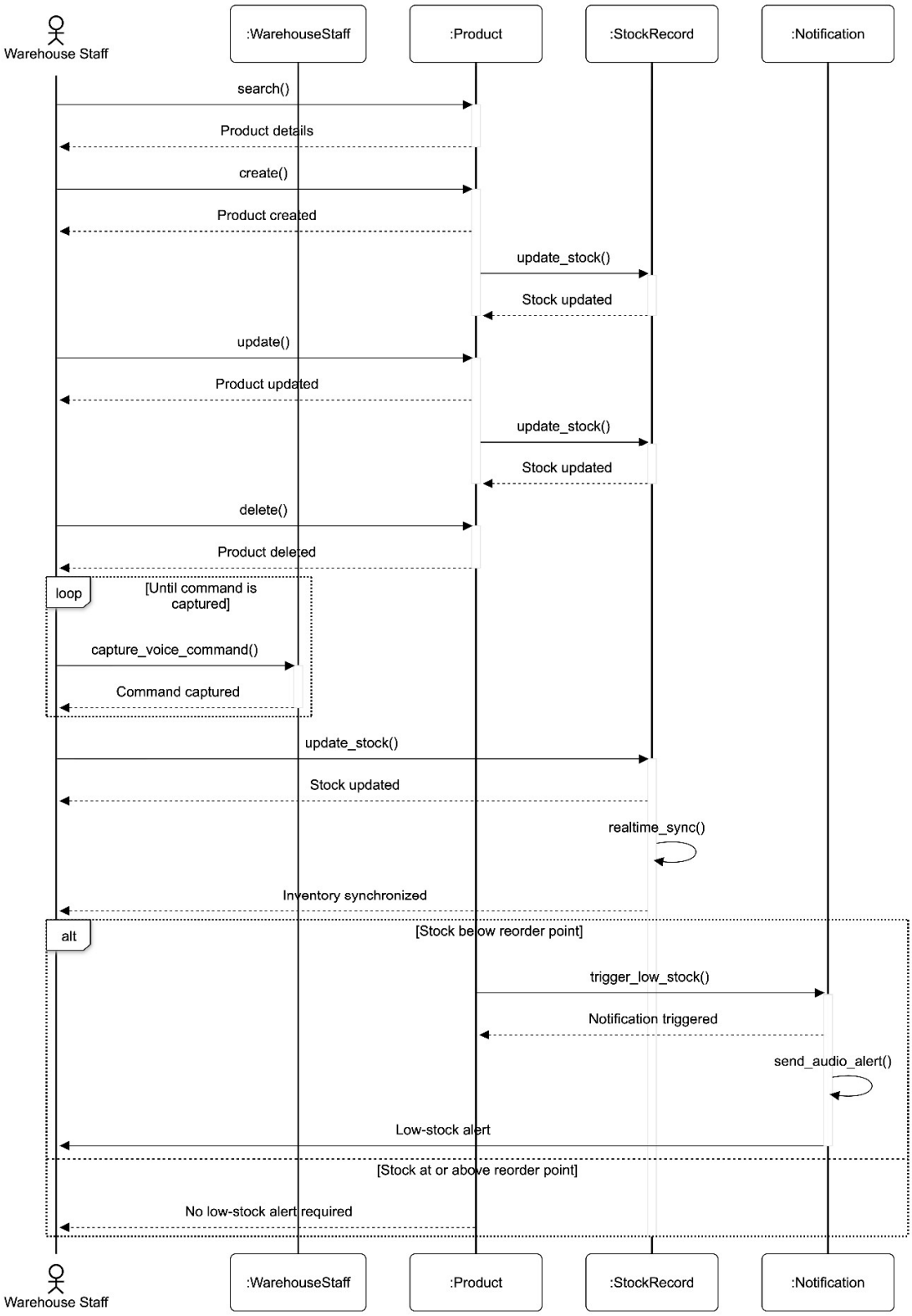
4.4 Diagram Consistency

For all three use cases, the participating classes, actors, and operations were kept consistent with the class diagram. The sequence diagrams show the chronological order of messages, while the corresponding collaboration diagrams represent the same interactions through numbered messages between connected objects. UML fragments and guard conditions are used to represent the alternative and optional behavior described in the SRS.

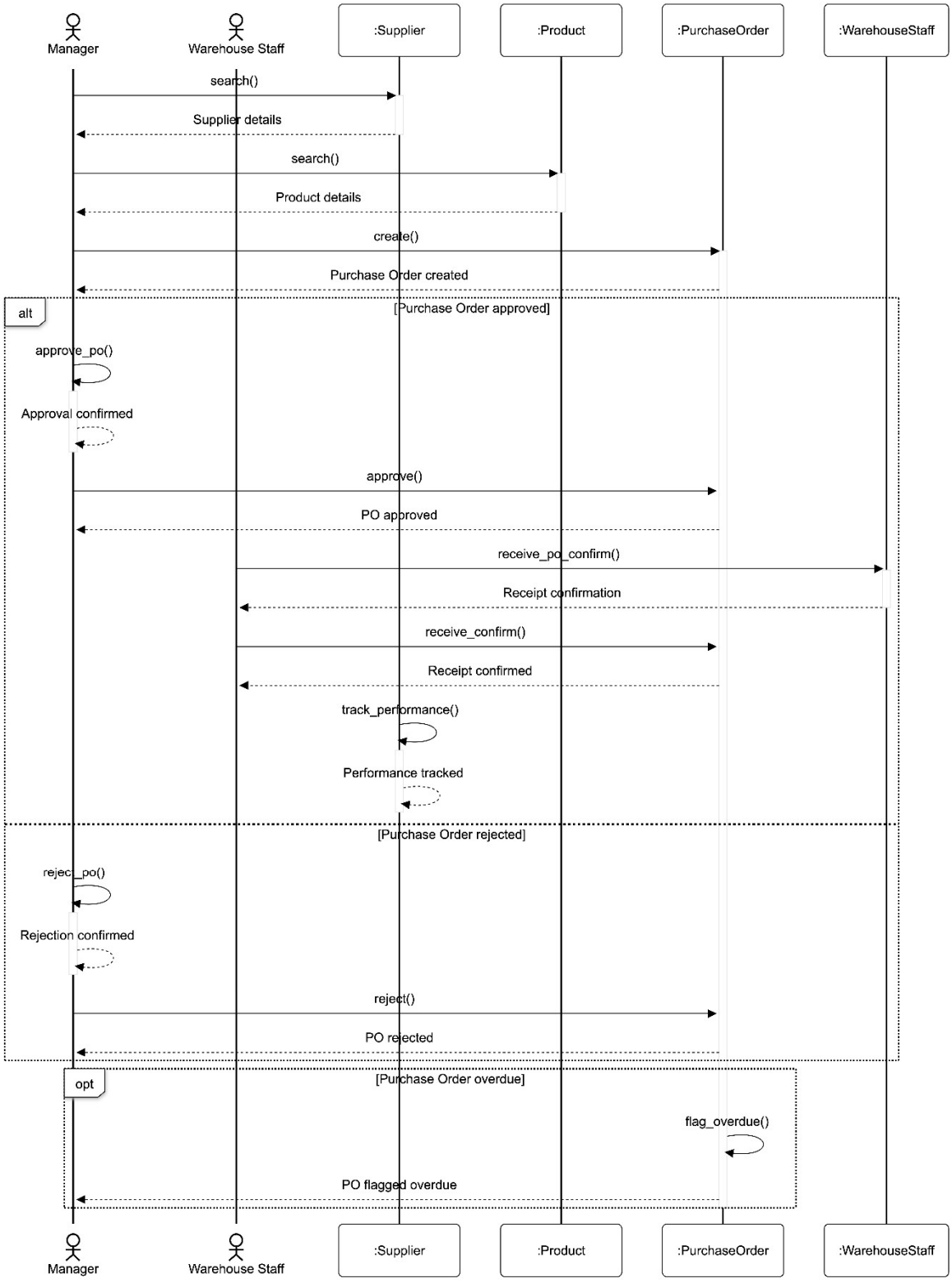
Thus, the implementation provides a dynamic view of Inventory Management, Manage Purchase Order, and Process Sales while preserving the relationship between the SRS requirements, class diagram, sequence diagrams, and collaboration diagrams.

Sequence and Collaboration Diagrams:

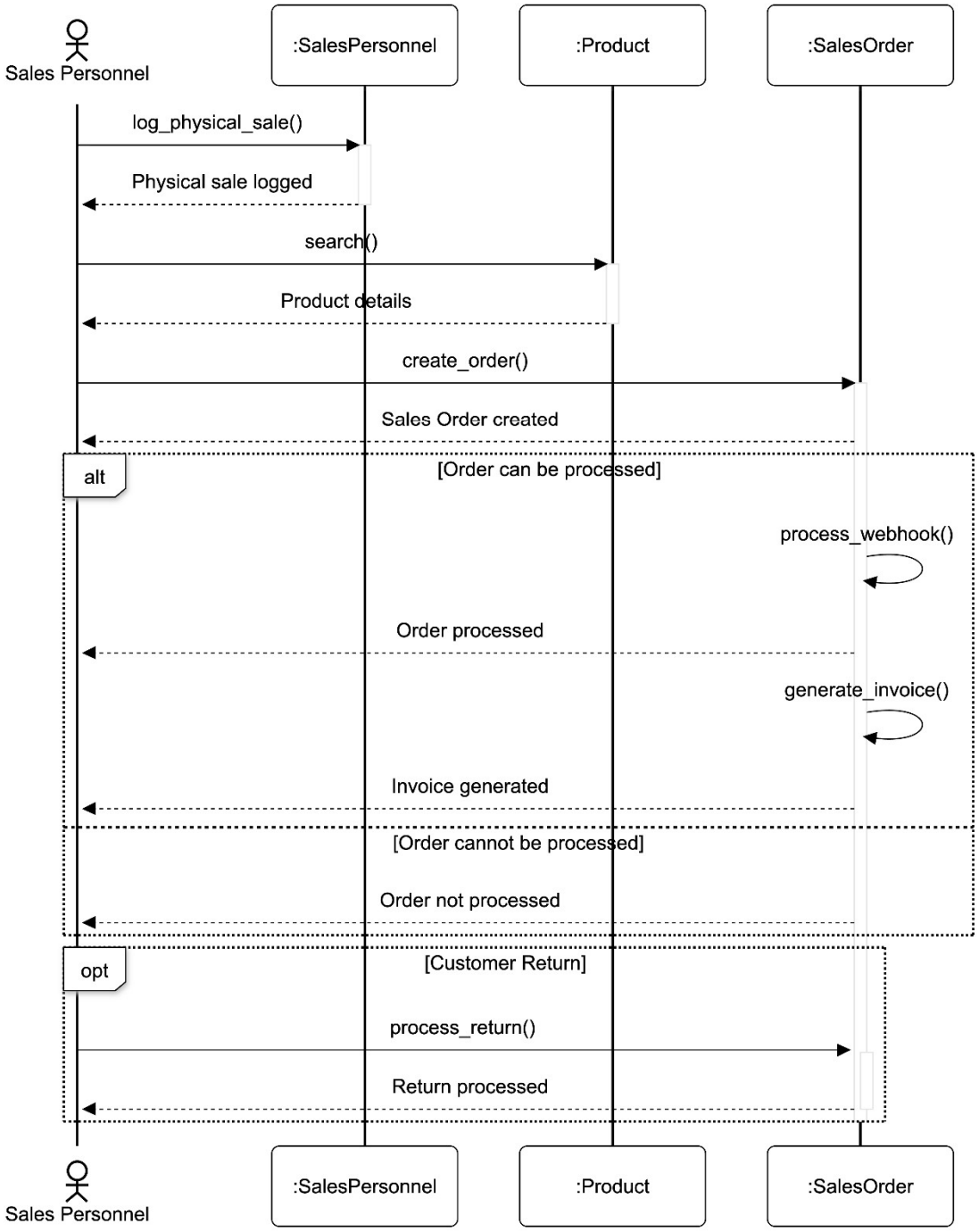
1. **Inventory Management**


Warehouse Staff
:WarehouseStaff
:Product
:StockRecord
:Notification
search()
Product details
create()
Product created
update_stock()
Stock updated
update()
Product updated
update_stock()
Stock updated
delete()
Product deleted
loop
[Until command is captured]
capture_voice_command()
Command captured
update_stock()
Stock updated
realtime_sync()
Inventory synchronized
alt
[Stock below reorder point]
trigger_low_stock()
Notification triggered
send_audio_alert()
Low-stock alert
[Stock at or above reorder point]
No low-stock alert required
Warehouse Staff
:WarehouseStaff
:Product
:StockRecord
:Notification

2. Manage Purchase Order



3. Process Sales



Conclusion:

The Sequence and Collaboration Diagrams provide a dynamic representation of the Inventory and Stock Management System by showing how actors and system objects communicate while executing important use cases.

The sequence diagrams represent the order and timing of interactions, while the collaboration diagrams represent the relationships and message flow between participating objects. The selected scenarios—physical sales processing, purchase order management, and authentication—cover major functional areas of the ISMS and are supported by the requirements specified in the SRS.

The diagrams are derived from the classes and methods identified in the class diagram, ensuring consistency between the structural and behavioral models. The use of UML fragments such as `alt` and `opt` also allows conditional and optional behavior defined in the SRS to be represented clearly. Overall, these diagrams provide a clear understanding of how the ISMS performs its major operations and how different users and system objects cooperate to achieve each use case.